

Mutable vs Immutable Objects in Python

- There are two types of objects in Python i.e. **Mutable** and **Immutable objects**.

Immutable Objects

- Immutable Objects are of in-built datatypes like int, float, bool, string (str) and tuple. In simple words, an immutable object can't be changed after it is created.

```
# Python code to test that  
# tuples are immutable
```

```
tuple1 = (0, 1, 2, 3)  
tuple1[0] = 4  
print(tuple1)
```

SMP

Traceback (most recent call last):

File "e0eaddff843a8695575daec34506f126.py", line
3, in

```
    tuple1[0]=4
```

TypeError: 'tuple' object does not support item
assignment

CHINTECH¹

Mutable vs Immutable Objects in Python

Example 2: We will take a Python string and try to modify its value. Similar to the tuple, strings are immutable and will throw an error.

```
# Python code to test that  
# strings are immutable  
  
message = "Welcome"  
message[0] = 'p'  
print(message)
```

```
Traceback (most recent call last):  
  File  
    "/home/ff856d3c5411909530c4d328eeca165b.py",  
    line 3, in  
      message[0] = 'p'  
TypeError: 'str' object does not support item  
assignment
```

Mutable vs Immutable Objects in Python

Mutable Objects

- Mutable Objects are of type Python **list**, Python **dict**, or Python **set**. Custom classes are generally mutable.

Example 1: Add and Remove items from a list in Python

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list)

my_list.insert(1, 5)
print(my_list)

my_list.remove(2)
print(my_list)

popped_element = my_list.pop() #remove last element
print(my_list)
print(popped_element)
```

```
[1, 2, 3, 4]
[1, 5, 2, 3, 4]
[1, 5, 3, 4]
[5, 3, 4]
4
```